



**TECNOLÓGICO
NACIONAL DE MÉXICO**



Tópicos de Inteligencia Artificial

Tarea:

Sistema de Riego

Alumno:

Rementería Medina Jesús Hector

Descripción del Problema

Se requiere de optimizar la colocación de sensores en un campo agrícola para maximizar la eficiencia del riego mediante el uso de un algoritmo de enjambre de partículas bioinspiradas (PSO), el algoritmo debe encontrar la mejor distribución de sensores teniendo en cuenta las siguientes variables:

- Topografía del terreno
- Distribución de cultivos
- Variabilidad de la humedad del suelo

El objetivo es optimizar la ubicación de sensores en un campo agrícola para maximizar la eficiencia del riego

Justificación

El problema a solucionar corresponde a un problema de optimización multidimensional, ya que tenemos múltiples variables que intervienen como podrían ser la condición del suelo, la distribución de los cultivos y las características del terreno, esto causa que exista numerosas combinaciones posibles dentro del área de estudio

Seleccionamos el uso del algoritmo PSO porque nos permite explorar eficientemente espacios de búsqueda complejos y encontrar soluciones cercanas al óptimo global, este algoritmo cuenta con las siguientes ventajas que nos serán útiles para resolver nuestro problema:

- Es sencillo de implementar a diferencia de otros algoritmos evolutivos
- Tiene rápida convergencia hacia soluciones óptimas
- Es adecuado para resolver problemas continuos o multidimensionales
- Permite trabajar con restricciones de espacio y límites geográficos
- Tiene una buena capacidad de exploración y adaptación entre diferentes configuraciones del terreno

además de que este algoritmo tiene la capacidad de manejar escenarios dinámicos y escalables que nos permitiría aumentar la cantidad de cultivos, sensores o datos geográficos, todas estas características nos llevan a conclusión de que el uso de PSO nos será de gran ayuda para resolver nuestro problema

Diseño del Algoritmo

Haremos uso del algoritmo Particle Swarm Optimizacion (PSO) para determinar nuestra distribución optima de los sensores, tomando en cuenta las variables de humedad del suelo, salinidad, elevación del terreno y distribución espacial de cultivo, las partículas de nuestro algoritmo representaran a los sensores y su ubicación por lo que cada sensor estará definido mediante

(latitud, Longitud) o también podría ser $(x_1, y_1, x_2, y_2, \dots, x_n, y_n)$

Donde N representa nuestra cantidad de sensores y cada (x, y) será la posición de cada sensor

El algoritmo comienza generando un conjunto de partículas en posiciones aleatorias, pero asegurando que no se salgan de los límites geográficos, cada partícula cuenta con la posición actual, la velocidad y la mejor posición individual encontrada, y el sistema mantiene una mejor solución global compartida

La función Fitness evalúa la configuración de los sensores considerando la cobertura del terreno, esto mediante el cálculo de la distancia entre cada punto agrícola y el sensor más cercano, mientras mayor sea la distancia mayor será el costo de la solución

En cuanto a la humedad se considera su desviación estándar para identificar zonas con comportamiento irregular, las zonas con mayor salinidad generan penalizaciones, mientras que la variación topográfica afecta la distribución de la humedad

También se toma en cuenta la distancia que hay entre los sensores asegurándose que no se coloquen muy cerca para evitar redundancia y mejorar la cobertura del terreno

Parámetros del algoritmo

Número de partículas: 40

Iteraciones: 100

Coeficiente cognitivo (c_1): 1.5

Coeficiente social (c_2): 1.5

Inercia (w): 0.7

Implementación

```
# Funcion Fitness
def fitness_function(particulas): 1 usage
    n_particulas = particulas.shape[0]
    scores = []
    for i in range(n_particulas):
        particula = particulas[i]
        PosSensores = particula.reshape(Sensores, 2)
        score = 0
        #Cobertura del terreno
        for punto in coords:
            distancias = np.linalg.norm(
                PosSensores - punto,
                axis=1
            )
```

```
            min_dist = np.min(distancias)
            score += min_dist * 10
        #Variabilidad de humedad
        score += np.std(humedad) * 5
        #Penalización por salinidad
        score += np.mean(salinidad) * 10
        #Variación topográfica
        score += np.std(df["elevacion"]) * 2
        #Evitar sensores pegados
        for j in range(Sensores):
            for k in range(j + 1, Sensores):
                d = np.linalg.norm(
                    PosSensores[j] - PosSensores[k]
                )
                if d < 0.01:
                    score += 50
        scores.append(score)
    return np.array(scores)
```

Esta es la función Fitness de nuestro programa, recibe como parámetro todas las partículas actuales que contienen las soluciones generadas por el algoritmo PSO,

Posteriormente obtiene la cantidad total de partículas para ser evaluadas individualmente, se crea una lista vacía que almacenara el costo fitness de cada partícula, luego dentro de for recorre cada una de las partículas para evaluar su calidad

Primero obtiene la partícula actual a analizar, y recupera los datos sobre la posición de dicha partícula, inicializa el costo como 0 y recorre todos los puntos de nuestro Dataset, calcula la distancia que hay entre los puntos agrícolas y los sensores utilizando distancia euclidiana

Obtiene la distancia menor y realiza las penalizaciones correspondientes según varios factores, la distancia entre el sensor y los puntos, la humedad, la salinidad del terreno, y la elevación del terreno, por ultimo comprueba que los sensores no se encuentren demasiado cerca calculando la distancia entre ellos y penaliza si se encuentran demasiado cerca para evitar redundancias

Por último, la función devuelve un arreglo con el costo de cada una de las partículas evaluadas esto para que nuestro PSO utilice estos valores para decidir cuales soluciones son mejores o como realizar los movimientos para próximas iteraciones

```
# Configuración
Configuración = {
    'c1': 1.5, #Coeficiente propio
    'c2': 1.5, #Coeficiente social
    'w': 0.7   #Inercia
}
```

Aquí definimos la configuración que usaremos colocando los valores del coeficiente propio, coeficiente social y la inercia

```
# límites Geograficos
lat_min, lat_max = df["latitud"].min(), df["latitud"].max()
lon_min, lon_max = df["longitud"].min(), df["longitud"].max()

Límites = (
    np.array([lat_min, lon_min] * Sensores),
    np.array([lat_max, lon_max] * Sensores)
)
```

Definimos los límites del área utilizando la altitud y la longitud máxima del Dataset

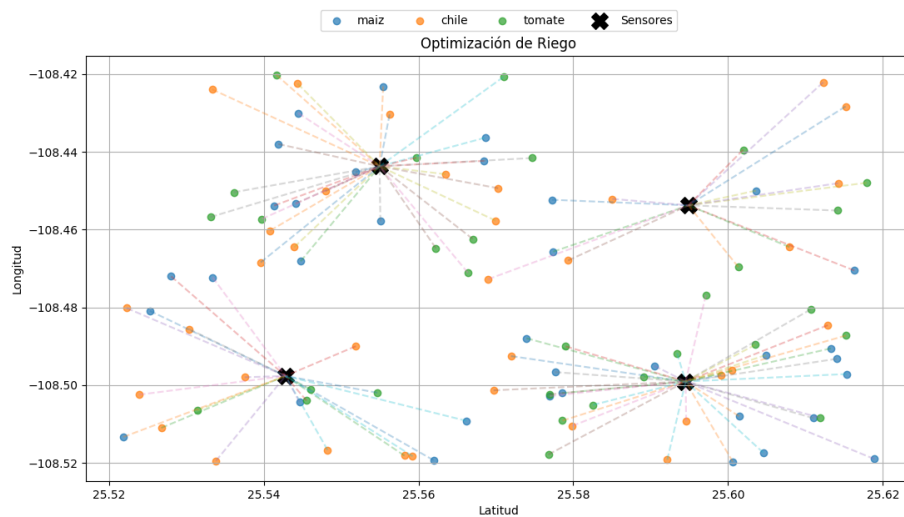
```
optimizer = ps.single.GlobalBestPSO(
    n_particles=40,
    dimensions=Sensores * 2,
    options=Configuracion,
    bounds=Limites
)
```

Aquí creamos una instancia del algoritmo PSO utilizando la librería de PySwarms, funciona haciendo que todas las partículas del enjambre compartan la mejor solución global encontrada, posteriormente definimos la cantidad de partículas para nuestro enjambre, también definimos la cantidad de dimensiones que tendrán en nuestro caso son 2 ya que contamos con latitud y longitud, por ultimo le mandamos la configuración y los límites que definimos anteriormente

```
#Optimizacion
best_cost, best_pos = optimizer.optimize(fitness_function, iters=100)
PosSensores = best_pos.reshape(Sensores, 2)
```

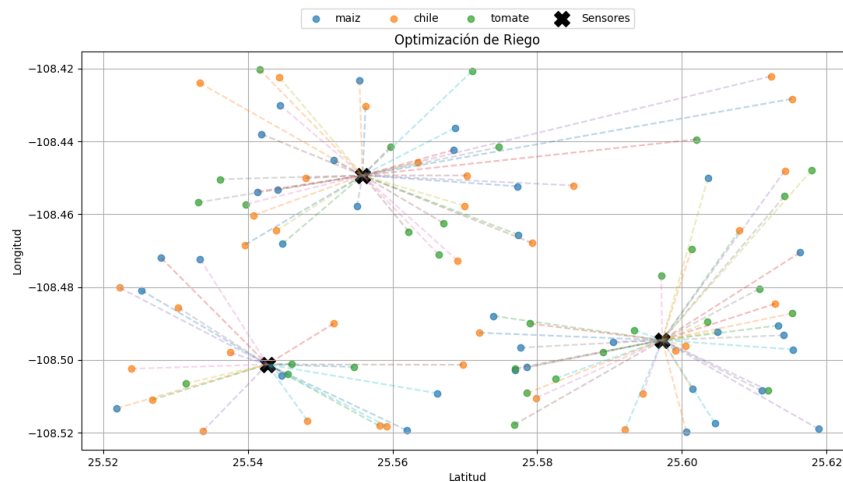
Finalmente llevamos a cabo la optimización realizando 100 iteraciones, durante estas interacciones se van a estar generando las posiciones de los sensores para luego evaluar cada solución mediante nuestra función fitness, compara los resultados y actualiza las velocidades y mueve las partículas así buscando durante las 100 iteraciones buscando las soluciones mejores

Ejemplos



```
----- RESULTADOS -----|  
Mejor costo: 119.54341432826796  
Sensores mas óptimos:  
[[ 25.59487771 -108.45375869]  
 [ 25.55496715 -108.44372362]  
 [ 25.59451967 -108.4990535 ]  
 [ 25.54280336 -108.49761073]]
```

Usando 4 Sensores

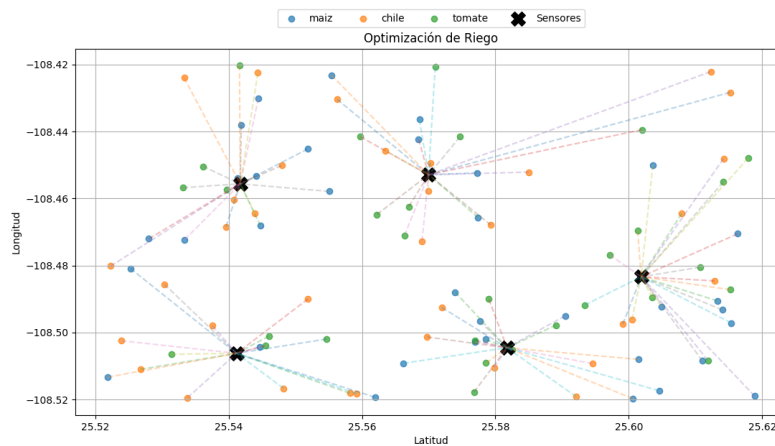


```

----- RESULTADOS -----
Mejor costo: 122.19613754762683
Sensores mas óptimos:
[[ 25.55586401 -108.44934091]
 [ 25.54268637 -108.50123411]
 [ 25.59727816 -108.49457961]]

```

Usando 3 Sensores

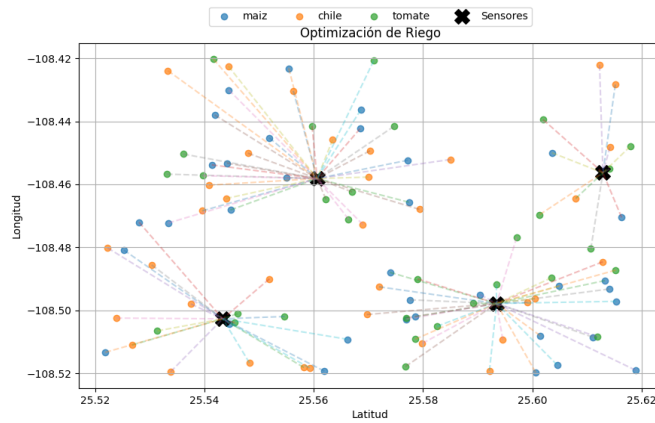


```

----- RESULTADOS -----
Mejor costo: 117.46247382165049
Sensores mas óptimos:
[[ 25.5817289 -108.50452698]
 [ 25.56998008 -108.4529164 ]
 [ 25.54173852 -108.45561863]
 [ 25.54118316 -108.50619708]
 [ 25.60186515 -108.4833528 ]]

```

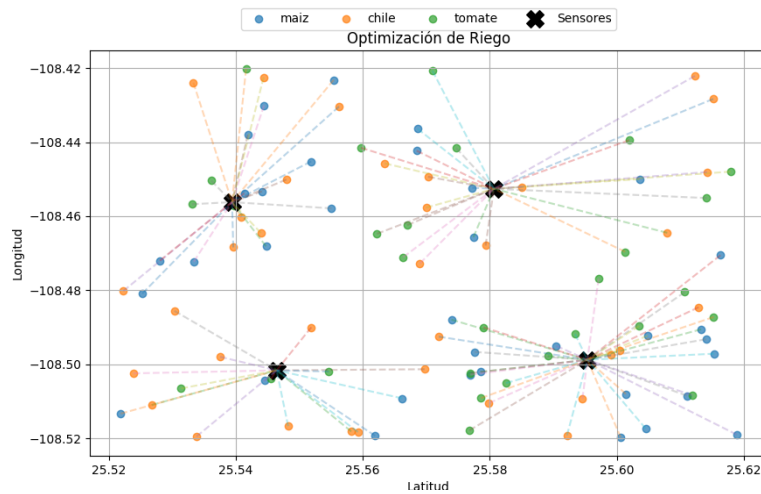
Usando 5 Sensores



```

----- RESULTADOS -----
Mejor costo: 129.92171865736302
Sensores mas óptimos:
[[ 25.61291867 -108.45633018]
 [ 25.56061134 -108.45816944]
 [ 25.59345522 -108.4978012 ]
 [ 25.54334845 -108.50266685]]
  
```

Usando 4 Sensores y Alta Salinidad



```

----- RESULTADOS -----
Mejor costo: 164.52383185329845
Sensores mas óptimos:
[[ 25.58064781 -108.45260975]
 [ 25.5464285 -108.50160821]
 [ 25.53940614 -108.45620401]
 [ 25.59536865 -108.49876185]]
  
```

Usando 4 Sensores y alta humedad

Resultados

Gracias al uso del algoritmo PSO se logró encontrar las distribuciones más eficientes de sensores dentro del terreno analizado, las simulaciones realizadas utilizando distantes cantidades de sensores mostraron que la cantidad de sensores más eficiente es de 4 dada la cantidad de puntos, se demostraron mejoras en la cobertura del terreno y la reducción de distancias, además el sistema pudo adaptarse a distintos escenarios con una salinidad y humedad diferente sin perder estabilidad

Análisis de Eficiencia

El uso de 40 partículas permitió explorar múltiples soluciones simultáneamente sin tener tiempos de ejecución elevados, las penalizaciones lograron evitar la redundancia entre los sensores y mejorar la cobertura del terreno, comparado con las distribuciones aleatorias, el PSO obtuvo resultados eficientes y mejor adaptados a las condiciones del dataset

Conclusiones

Este proyecto permitió comprobar que el algoritmo PSO es una herramienta eficiente para la optimización de sensores de humedad en terrenos agrícolas, logro considerar variables importantes como la humedad, salinidad, topografía y distribución de cultivos para encontrar soluciones funcionales, el algoritmo demostró una capacidad de adaptación y estabilidad ante diferentes escenarios del terreno